Mocking in Python

Ivan Ogasawara

2026-06-11

A mock is a controlled replacement for something your code normally uses.

In Python, we often use mocks in tests. Instead of letting the code talk to a real database, call a real web API, send a real email, or read a real file, we replace that dependency with a small object or function that behaves in a predictable way.

Mocks are useful because they let us test one piece of code at a time. The goal is not to test the internet, a third-party API, or a database server. The goal is to check whether our own function behaves correctly when it receives a certain response.

When to Use a Mock

Use a mock when the real dependency is:

- Slow, such as a network request.
- Unstable, such as an external API that may be temporarily unavailable.
- Expensive, such as a paid service.
- Dangerous, such as code that sends emails, writes files, or changes production data.
- Hard to reproduce, such as an API error or timeout.

Mocks are especially helpful when a test should always produce the same result. If a test depends on a live website, it may pass today and fail tomorrow even when your code did not change.

When Not to Use a Mock

Do not mock everything. If a function is simple and does not depend on an external system, it is usually better to test it directly.

For example, a function that calculates a total price does not need a mock:

```
def total_price(price, quantity):
    return price * quantity
```

Mocks are most useful at the boundary between your code and something outside your code.

Example: A Function That Calls an API

Imagine we have this function. It calls an API with `requests.get`, reads the JSON response, and returns the user's name:

```
import requests


def get_user_name(user_id):
```

```
    url = f"https://api.example.com/users/{user_id}"
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
    return data["name"]
```

Testing this function against the real API would make the test depend on the network. The API might be down, the user might not exist anymore, or the response might change.

Instead, we can replace `requests.get` with our own function during the test.

Creating a Mock Response

The real `requests.get` returns a response object. Our function only needs two methods from that object:

- `raise_for_status()`
- `json()`

So our mock response only needs to implement those two methods:

```
class MockResponse:
    def __init__(self, json_data, status_code=200):
        self.json_data = json_data
        self.status_code = status_code

    def json(self):
        return self.json_data

    def raise_for_status(self):
        if self.status_code >= 400:
            raise Exception(f"HTTP error: {self.status_code}")
```

Now we can create a mock version of `requests.get`:

```
def mock_requests_get(url, timeout=None):
    return MockResponse({"id": 1, "name": "Ada Lovelace"})
```

This function accepts `url` and `timeout` because the original code calls `requests.get(url, timeout=5)`. A mock should accept the arguments that the real function receives.

Activating and Deactivating the Mock

One simple way to activate a mock is to temporarily replace `requests.get`.

```
import requests


original_requests_get = requests.get


def activate_requests_get_mock():
    requests.get = mock_requests_get
```

```
def deactivate_requests_get_mock():
    requests.get = original_requests_get
```

After `activate_requests_get_mock()` runs, any code that calls `requests.get` will call `mock_requests_get` instead.

After `deactivate_requests_get_mock()` runs, `requests.get` is restored to its original behavior.

Using the Mock in a Test

Here is the full example:

```
import requests


def get_user_name(user_id):
    url = f"https://api.example.com/users/{user_id}"
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
    return data["name"]


class MockResponse:
    def __init__(self, json_data, status_code=200):
        self.json_data = json_data
        self.status_code = status_code

    def json(self):
        return self.json_data

    def raise_for_status(self):
        if self.status_code >= 400:
            raise Exception(f"HTTP error: {self.status_code}")


def mock_requests_get(url, timeout=None):
    return MockResponse({"id": 1, "name": "Ada Lovelace"})


original_requests_get = requests.get


def activate_requests_get_mock():
    requests.get = mock_requests_get


def deactivate_requests_get_mock():
    requests.get = original_requests_get
```

```
def test_get_user_name():
    activate_requests_get_mock()

    try:
        result = get_user_name(1)
        assert result == "Ada Lovelace"
    finally:
        deactivate_requests_get_mock()
```

The `try` and `finally` block is important. It makes sure the mock is deactivated even if the test fails.

Without `finally`, a failed test could leave `requests.get` mocked. That can cause confusing failures in other tests.

Checking the Request

Sometimes we also want to check whether our function called the correct URL. We can store the URL inside a list:

```
calls = []


def mock_requests_get(url, timeout=None):
    calls.append({"url": url, "timeout": timeout})
    return MockResponse({"id": 1, "name": "Ada Lovelace"})
```

Then the test can check both the result and the request:

```
def test_get_user_name():
    activate_requests_get_mock()

    try:
        result = get_user_name(1)

        assert result == "Ada Lovelace"
        assert calls[0]["url"] == "https://api.example.com/users/1"
        assert calls[0]["timeout"] == 5
    finally:
        deactivate_requests_get_mock()
```

This is one of the main benefits of mocks: we can test what our code returns and how it interacts with its dependencies.

A Safer Pattern

Manually replacing `requests.get` is useful for learning how mocks work. In larger projects, Python's `unittest.mock` module is usually safer because it automatically restores the original function.

The same test can be written like this:

```
from unittest.mock import patch
```

```
def test_get_user_name():
    with patch("requests.get", mock_requests_get):
        result = get_user_name(1)

    assert result == "Ada Lovelace"
```

The `with` block activates the mock. When the block ends, Python deactivates the mock automatically.

Key Idea

A mock is a replacement that gives your test control.

For `requests.get`, a mock lets us test code that depends on an HTTP response without making a real HTTP request. The test becomes faster, more stable, and easier to understand.